

《基础软件开发技术实践——智能软件工程》实验指导书

1 实验一 代码审查意见挖掘与需求理解

1.1 实验基本信息

实验名称	代码审查意见挖掘与需求理解
实验学时	4 学时
实验类型	综合性实验
实验性质	必修
教学方式	讲授 + 实践
对应课程目标	T1、T2

1.2 实验目的

完成本实验后，学生应能够：

1. 理解现代软件开发过程中 Pull Request (PR) 与 Code Review 的基本流程。
2. 熟悉 GitHub 开源软件仓库中代码审查数据的组织形式。
3. 掌握 Pull Request、Commit、Review、Comment 等对象之间的关系。
4. 能够利用 GitHub API 或公开数据集获取代码审查数据。
5. 能够完成代码、文本及元数据等多源信息的抽取。
6. 掌握数据集统计分析方法，为后续模型训练提供数据基础。

1.3 实验背景

随着 GitHub、GitLab 等协同开发平台的广泛应用，代码审查（Code Review）已经成为现代软件开发过程中保证软件质量的重要环节。开发人员通常通过 Pull Request (PR) 的形式提交代码修改，并由项目维护者或其他开发人员进行审查、讨论和修改建议，最终决定是否将代码合并到主分支。

近年来，大语言模型（Large Language Models, LLMs）的快速发展进一步改变了代码审查过程。一方面，大量代码已经由 GitHub Copilot、ChatGPT、Claude Code、Cursor 等工具辅助生成；另一方面，越来越多的代码审查意见也开始由 AI 自动生成。因此，现代代码审查已经逐渐形成了“人类开发者 + AI 开发者 + AI Reviewer”共同参与的软件开发模式。

为了研究不同代码审查方法的性能，本课程首先需要构建一个包含丰富信息的数据集，包括：

- Pull Request 基本信息；
- 提交代码；
- Code Review Comments；
- Review Decision；
- Merge 状态；
- 是否包含 AI 生成代码；
- 是否存在 AI Reviewer；
- 时间、标签等元数据。

该数据集将作为整个课程后续所有实验的数据基础。

1.4 实验原理

1.4.1 Pull Request 工作流程

现代开源软件通常采用 Pull Request 的开发流程，如图所示。

此处插入 Pull Request 工作流程图

Fork Repository

↓

Commit Changes

↓

Open Pull Request

↓

Code Review

↓

Discussion

↓

Merge / Close

Pull Request 包含的不仅仅是代码修改，还包括：

- Commit 信息；
- 修改文件；
- Issue 引用；
- Code Review；
- Discussion；
- Merge Decision。

1.4.2 代码审查流程

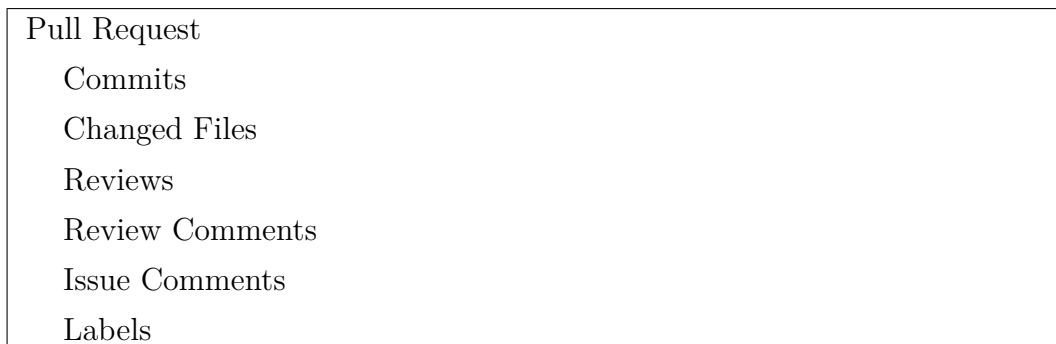
代码审查主要包含以下几个阶段：

1. 提交代码修改；
2. Reviewer 阅读代码；
3. Reviewer 提交修改建议；
4. Developer 回复意见；
5. 修改代码；
6. 最终决定是否 Merge。

代码审查不仅能够发现软件缺陷，还能够提高代码可维护性、统一编码规范，并促进团队成员之间的知识共享。

1.4.3 Pull Request 数据结构

一次 Pull Request 通常包含多个对象，其关系如图所示。



这些对象共同构成了代码审查任务的重要数据来源。

1.4.4 代码审查任务

本课程主要关注两个典型任务：

1. Merge Prediction

判断一个 Pull Request 是否最终能够被项目接受。

2. Review Comment Generation

根据代码修改内容自动生成合理的代码审查意见。

后续实验均围绕这两个任务展开。

1.5 实验环境

1.5.1 硬件环境

建议配置如下：

- CPU：4 Core 以上；
- 内存：8GB 以上；
- 推荐网络连接 GitHub。

1.5.2 软件环境

建议软件版本如下：

软件	推荐版本
Python	3.10+
Git	Latest
VSCode	Latest
Jupyter Notebook	Latest
pandas	最新版本
requests	最新版本
PyGithub	最新版本
matplotlib	最新版本

1.6 实验内容

本实验需要完成以下任务：

1. 熟悉 GitHub Pull Request 的组织形式；
2. 获取指定开源项目的 Pull Request 数据；
3. 获取 Pull Request 对应 Review；
4. 获取 Code Review Comments；
5. 获取 Commit 信息；
6. 获取 Merge 状态；
7. 提取 Pull Request 标签；
8. 判断是否存在 AI Reviewer；
9. 判断是否可能由 AI 生成代码；
10. 对整个数据集进行统计分析。

1.7 实验步骤

1.7.1 步骤一：获取 Pull Request

选择 5 个大型开源软件，利用 GitHub API 对每个项目获取 300 条 PR。不要求每条 PR 同时有 AI 生成代码和 AI reviewer^[F]

- PR ID
- Title
- Body
- Author
- Created Time
- Merge Status

1.7.2 步骤二：获取 Code Review

继续获取：

- Reviewer
- Review Decision
- Review Time
- Review Comment

1.7.3 步骤三：获取代码修改

提取：

- 修改的文件
- 修改的函数
- Diff
- Commit Message

1.7.4 步骤四：提取特征

完成如下信息统计：

- PR 长度
- 修改文件数
- Reviewer 数量
- Comment 数量

- 是否 Merge
- Label 数量
- 是否包含 AI Reviewer
- 是否存在 AI Generated Code

1.7.5 步骤五：数据分析

完成统计分析，包括但不限于：

- Merge 与 Non-Merge 数量统计；
- Review Comment 数量分布；
- Label 分布；
- Reviewer 数量分布；
- PR 长度分布；
- AI 与 Human PR 数量比较。

建议绘制柱状图、饼图、直方图等可视化结果。

1.8 实验结果

实验报告中，学生应展示以下实验结果：

1. 成功获取的 Pull Request 数据样例；
2. 成功获取的 Review Comments 样例；
3. 成功完成数据预处理的阳历；
4. 数据统计结果；
5. 可视化分析结果。

1.9 实验思考

请结合实验过程回答以下问题：

- 1. Pull Request 与 Commit 有何区别？
- 2. 为什么 Code Review 能够提高软件质量？
- 3. Merge Prediction 可以应用于哪些实际场景？
- 4. AI Reviewer 与 Human Reviewer 的审查方式可能有哪些差异？
- 5. 数据集是否存在类别不平衡问题？如何解决？

1.10 实验拓展

完成基础实验后，可进一步尝试：

- 分析不同开源社区（如 Kubernetes、PyTorch、VSCode）之间代码审查流程的差异；
- 构建适用于整个课程的数据仓库，为后续机器学习和深度学习实验提供统一的数据输入。

2 实验二 代码审查意见挖掘与需求理解

2.1 实验基本信息

实验名称	基于机器学习的人类编写代码的代码审查
实验学时	4 学时
实验类型	综合性实验
实验性质	必修
教学方式	讲授 + 实践
对应课程目标	T1、T2

2.2 实验目的

完成本实验后，学生应能够：

- 1. 理解传统机器学习在代码审查任务中的基本流程。
- 2. 掌握针对代码审查任务的数据预处理方法。

3. 掌握代码抽象语法树（AST）和控制流图（CFG）的基本构建方法。
4. 能够提取代码结构特征、文本特征以及统计特征。
5. 掌握支持向量机（SVM）和随机森林（Random Forest）等传统机器学习算法。
6. 能够建立代码是否被合并（Merge Prediction）的分类模型，并分析模型性能。

2.3 实验背景

实验一完成了代码审查数据集的构建，为代码审查模型训练提供了基础数据。然而，原始的 Pull Request 数据不能直接用于机器学习模型训练，需要首先完成数据预处理和特征工程。

传统机器学习方法通常依赖人工设计的特征，通过提取代码结构、代码复杂度、文本信息以及开发过程中的统计信息，将代码修改表示为固定长度的特征向量，再利用分类模型判断 Pull Request 是否能够成功合并（Merge）。

虽然近年来深度学习和大语言模型已经广泛应用于代码理解任务，但传统机器学习方法仍具有训练成本低、模型可解释性强以及训练速度快等优点，因此仍然是代码审查领域的重要基线方法。本实验将基于实验一构建的数据集，训练传统机器学习模型，为后续深度学习模型提供性能比较基准。

2.4 实验原理

2.4.1 机器学习代码审查流程

传统机器学习代码审查通常包括以下几个步骤：

数据预处理

↓

特征提取

↓

特征向量构建

↓

模型训练

↓

Merge Prediction

2.4.2 代码中间表示

程序代码可以转换为不同的中间表示形式，从不同角度描述程序语义。

- 抽象语法树（Abstract Syntax Tree, AST）：描述程序语法结构；

- 控制流图 (Control Flow Graph, CFG): 描述程序执行流程;
- Diff 信息: 描述代码修改内容;
- Commit Message: 描述开发者修改意图。

不同表示能够提供互补的信息, 共同支持后续特征提取。

2.4.3 特征工程

本实验主要构建以下几类特征:

- 代码结构特征 (AST 节点数量、CFG 节点数量等);
- 代码修改特征 (新增代码行数、删除代码行数、修改文件数量等);
- 文本特征 (Title、Body、Commit Message 长度等)。

上述特征经过归一化处理后构成机器学习模型输入。

2.4.4 传统机器学习模型

本实验主要采用以下两种经典模型:

1. 支持向量机 (Support Vector Machine, SVM)

SVM 通过寻找最优分类超平面对不同类别进行划分, 适用于中小规模数据集。

2. 随机森林 (Random Forest)

随机森林由多个决策树组成, 通过集成学习提高模型泛化能力, 并能够分析各特征的重要程度。

2.5 实验环境

2.5.1 硬件环境

建议配置如下:

- CPU: 4 Core 以上;
- 内存: 8GB 以上。

2.5.2 软件环境

建议软件版本如下：

软件	推荐版本
Python	3.10+
scikit-learn	最新版本
pandas	最新版本
numpy	最新版本
tree-sitter	最新版本
networkx	最新版本
matplotlib	最新版本

2.6 实验内容

本实验需要完成以下任务：

1. 从实验一构建的数据集中筛选人类编写代码对应的 Pull Request；
2. 提取代码 AST 表示；
3. 提取代码 CFG 表示；
4. 提取代码统计特征和文本特征；
5. 构建机器学习训练数据集；
6. 分别训练 SVM 和 Random Forest 分类模型；
7. 对模型进行性能评估，并分析不同特征的重要程度。

2.7 实验步骤

2.7.1 步骤一：数据筛选

读取实验一构建的数据集，仅保留人类编写代码对应的 Pull Request，并完成数据清洗。

2.7.2 步骤二：代码中间表示生成

针对每个 Pull Request 中的代码修改：

- 构建 AST；

- 构建 CFG;
- 统计 AST 节点数量;
- 统计 CFG 节点数量。

2.7.3 步骤三：特征提取

提取以下特征：

- 修改文件数量;
- 新增代码行数;
- 删除代码行数;
- Commit Message 长度;
- Pull Request 描述长度;
- AST 相关特征;
- CFG 相关特征。

2.7.4 步骤四：模型训练

分别训练以下模型：

- Support Vector Machine (SVM);
- Random Forest。

预测目标为：

- Pull Request 是否最终 Merge。

2.7.5 步骤五：模型评估

采用测试集评估模型性能，包括但不限于：

- Accuracy;
- Precision;
- Recall;
- F1-score;
- ROC-AUC。

进一步分析 Random Forest 模型输出的特征重要性。

2.8 实验结果

实验报告中，学生应展示以下实验结果：

1. 人类代码数据集统计结果；
2. AST 和 CFG 生成样例；
3. 特征提取结果样例；
4. SVM 模型实验结果；
5. Random Forest 模型实验结果；
6. 不同特征的重要性分析；
7. 不同模型性能对比结果。

2.9 实验思考

请结合实验过程回答以下问题：

1. 为什么需要构建 AST 和 CFG 等代码中间表示？
2. 为什么传统机器学习需要人工设计特征？
3. SVM 和 Random Forest 分别适用于哪些应用场景？
4. 哪类特征对 Merge Prediction 贡献最大？为什么？
5. 与实验一相比，本实验的数据预处理增加了哪些步骤？

2.10 实验拓展

完成基础实验后，可进一步尝试：

- 尝试引入 XGBoost、LightGBM 等集成学习模型；
- 增加代码复杂度、代码质量等静态分析特征；
- 比较不同类型特征组合对模型性能的影响，为后续深度学习实验提供基线模型。

3 实验三 基于深度学习的人类编写代码的代码审查

3.1 实验基本信息

实验名称	基于深度学习的人类编写代码的代码审查
实验学时	4 学时
实验类型	综合性实验
实验性质	必修
教学方式	讲授 + 实践
对应课程目标	T1、T2

3.2 实验目的

完成本实验后，学生应能够：

1. 理解深度学习在代码审查任务中的基本原理。
2. 掌握 Code2Vec、CodeBERT 等代码表示学习模型的基本思想。
3. 掌握利用深度学习模型进行代码特征自动学习的方法。
4. 能够利用深度学习模型完成 Pull Request 是否被合并预测任务。
5. 能够利用深度学习模型自动生成代码审查意见。
6. 理解深度学习模型相比传统机器学习模型的优势与不足。

3.3 实验背景

实验二采用传统机器学习方法完成了代码审查任务，通过人工设计代码结构特征、文本特征和统计特征建立 Merge Prediction 模型。然而，传统机器学习模型依赖大量人工特征工程，难以充分学习程序的深层语义信息，其性能受特征设计质量影响较大。

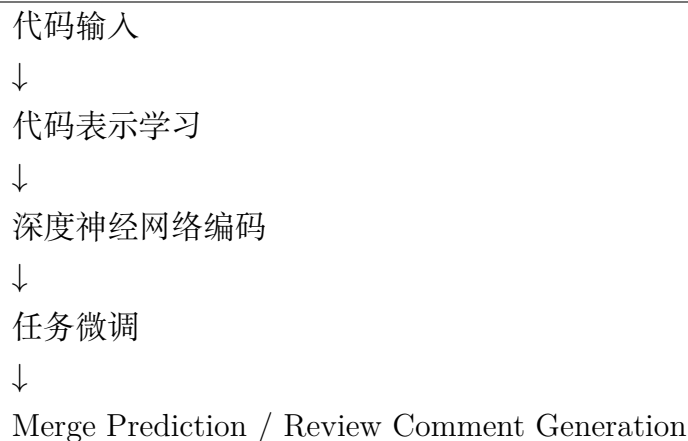
近年来，深度学习技术在智能软件工程领域得到了广泛应用。Code2Vec、Code2Seq、CodeBERT 等模型能够直接从程序源代码中学习语义表示，无需人工设计复杂特征，从而显著提高了代码理解能力。

本实验将在实验二构建的数据集基础上，利用 Code2Vec 和 CodeBERT 模型学习代码语义表示，并完成 Pull Request 是否能够被合并预测（Merge Prediction）和代码审查意见生成（Review Comment Generation）两个典型任务，为后续基于大语言模型的代码审查实验奠定基础。

3.4 实验原理

3.4.1 深度学习代码审查流程

深度学习代码审查通常包括以下几个步骤：



与传统机器学习不同，深度学习能够自动学习代码语义表示，无需人工设计大量特征。

3.4.2 Code2Vec 模型

Code2Vec 是一种基于抽象语法树路径（AST Paths）的代码表示学习模型。该模型首先从程序抽象语法树中提取大量路径，再利用注意力机制学习不同路径的重要程度，最终生成整个代码片段的向量表示。

Code2Vec 能够有效捕获程序的语法结构信息，适用于代码分类、代码克隆检测以及代码审查等任务。

3.4.3 CodeBERT 模型

CodeBERT 是一种面向源代码与自然语言联合训练的预训练模型。模型利用 Transformer 结构，通过海量开源代码进行预训练，能够学习程序语义和自然语言描述之间的对应关系。

由于 CodeBERT 已经学习了丰富的软件开发知识，因此能够较好地完成代码理解、代码搜索、代码摘要以及代码审查等任务。

3.4.4 代码审查任务

本实验主要完成以下两个任务：

1. Merge Prediction

根据 Pull Request 修改内容预测该 Pull Request 最终是否能够被项目接受并合并。

2. Review Comment Generation

根据 Pull Request 修改内容自动生成合理的代码审查意见，为开发人员提供修改建议。

3.5 实验环境

3.5.1 硬件环境

建议配置如下：

- CPU：8 Core 以上；
- 内存：16GB 以上；
- GPU：推荐 NVIDIA RTX3060 及以上。

3.5.2 软件环境

建议软件版本如下：

软件	推荐版本
Python	3.10+
PyTorch	最新版本
Transformers	最新版本
datasets	最新版本
pandas	最新版本
numpy	最新版本
matplotlib	最新版本

3.6 实验内容

本实验需要完成以下任务：

1. 从实验二构建的数据集中读取人类编写代码对应的数据；
2. 利用 Code2Vec 提取代码语义表示；
3. 利用 CodeBERT 学习代码表示；
4. 微调 CodeBERT 完成 Merge Prediction 任务；
5. 微调 CodeBERT 完成 Review Comment Generation 任务；
6. 比较不同深度学习模型的性能。

3.7 实验步骤

3.7.1 步骤一：数据准备

读取实验二处理完成的数据集，并划分训练集、验证集和测试集。

3.7.2 步骤二：Code2Vec 代码表示学习

利用 Code2Vec 模型生成代码向量表示，完成代码编码，并保存得到的向量表示结果。

3.7.3 步骤三：CodeBERT 模型微调

利用预训练 CodeBERT 模型完成模型微调，分别针对 Merge Prediction 任务和 Review Comment Generation 任务训练模型。

3.7.4 步骤四：模型预测

利用训练完成的模型，对测试集中的 Pull Request 进行预测，包括：

- 是否 Merge 预测；
- 自动生成代码审查意见。

3.7.5 步骤五：模型评估

采用测试集评估模型性能，包括但不限于：

- Accuracy;
- Precision;
- Recall;
- F1-score;
- BLEU (Review Comment Generation);
- ROUGE (Review Comment Generation)。

进一步比较 Code2Vec 与 CodeBERT 两种模型的实验结果。

3.8 实验结果

实验报告中，学生应展示以下实验结果：

1. Code2Vec 生成的代码向量样例；
2. CodeBERT 模型训练过程；
3. Merge Prediction 实验结果；
4. 自动生成的代码审查意见样例；
5. 不同深度学习模型性能比较结果；
6. 与实验二传统机器学习模型的性能对比分析。

3.9 实验思考

请结合实验过程回答以下问题：

1. 为什么深度学习模型能够减少人工特征工程？
2. Code2Vec 与 CodeBERT 分别学习了程序的哪些信息？
3. 为什么 CodeBERT 通常能够取得更好的实验效果？
4. Merge Prediction 与 Review Comment Generation 两个任务分别关注什么问题？
5. 深度学习模型相比传统机器学习模型有哪些优势与不足？

3.10 实验拓展

完成基础实验后，可进一步尝试：

- 尝试采用 GraphCodeBERT、UniXCoder 等最新代码预训练模型；
- 探索不同微调策略对模型性能的影响；
- 比较不同预训练模型在 Merge Prediction 和 Review Comment Generation 任务上的表现，为后续大语言模型代码审查实验提供基准。

4 实验四 基于大语言模型的人类编写代码的代码审查

4.1 实验基本信息

实验名称	基于大语言模型的人类编写代码的代码审查
实验学时	4 学时
实验类型	综合性实验
实验性质	必修
教学方式	讲授 + 实践
对应课程目标	T1、T2

4.2 实验目的

完成本实验后，学生应能够：

1. 理解大语言模型在代码审查任务中的应用原理。
2. 掌握 Prompt Engineering 的基本思想与设计方法。
3. 理解不同代码上下文对大语言模型性能的影响。
4. 能够利用大语言模型完成 Merge Prediction 任务。
5. 能够利用大语言模型自动生成代码审查意见。
6. 能够分析不同 Prompt 设计和上下文信息对代码审查结果的影响。

4.3 实验背景

实验三采用深度学习模型完成了代码表示学习和代码审查任务，但模型通常需要大量标注数据进行训练，并且只能针对固定任务进行优化。近年来，大语言模型（Large Language Models, LLMs）在代码理解、代码生成和软件工程任务中展现出了优异性能，使代码审查逐渐由传统监督学习转向基于提示词的推理模式。

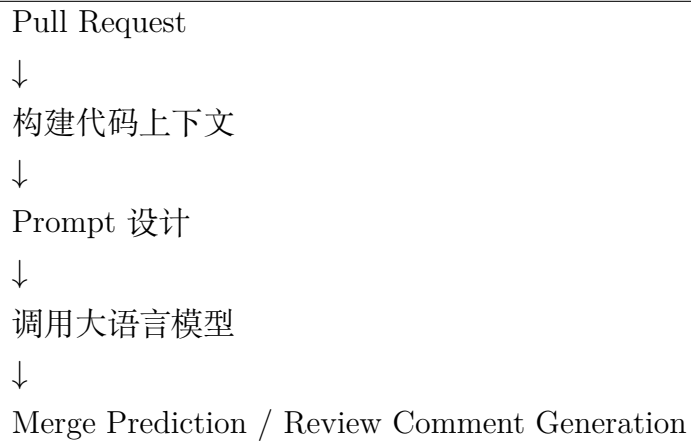
不同于 CodeBERT 等需要重新训练或微调的模型，大语言模型能够利用已有知识直接完成代码审查任务。然而，大语言模型的性能高度依赖输入信息的组织方式。合理选择代码上下文以及设计高质量 Prompt，能够显著提高代码审查质量。

本实验将在实验三构建的数据集基础上，调用开源或商用大语言模型，通过设计不同的代码上下文和提示词，完成 Merge Prediction 和 Review Comment Generation 两个任务，并分析不同设计策略对模型性能的影响。

4.4 实验原理

4.4.1 基于大语言模型的代码审查流程

大语言模型代码审查通常包括以下几个步骤：



与深度学习模型相比，大语言模型无需重新训练即可完成代码审查任务。

4.4.2 代码上下文

代码审查不仅依赖修改代码本身，还需要结合丰富的软件工程上下文。本实验主要考虑以下几类上下文：

- Pull Request 描述；
- Commit Message；
- 修改前后的 Diff；
- 修改文件内容；
- 历史代码片段；
- 历史代码审查意见。

不同上下文能够为模型提供不同层次的软件工程信息，从而影响最终审查结果。

4.4.3 Prompt Engineering

Prompt Engineering 是影响大语言模型性能的重要因素。

本实验主要探索以下 Prompt 设计方式：

- Zero-shot Prompt；
- Few-shot Prompt；

- Chain-of-Thought Prompt;
- Role-based Prompt。

不同 Prompt 能够引导模型采用不同的推理方式完成代码审查任务。

4.4.4 代码审查任务

本实验主要完成以下两个任务：

1. Merge Prediction

根据 Pull Request 修改内容及相关上下文预测是否能够成功 Merge。

2. Review Comment Generation

根据 Pull Request 生成完整、合理的代码审查意见，并分析不同 Prompt 设计对生成质量的影响。

4.5 实验环境

4.5.1 硬件环境

建议配置如下：

- CPU：8 Core 以上；
- 内存：16GB 以上；
- 网络环境：能够访问所使用的大语言模型服务。

4.5.2 软件环境

建议软件版本如下：

软件	推荐版本
Python	3.10+
Transformers	最新版本
OpenAI SDK 或兼容 SDK	最新版本
pandas	最新版本
numpy	最新版本
matplotlib	最新版本

4.6 实验内容

本实验需要完成以下任务：

1. 读取实验三构建的人类代码数据集；
2. 构建不同粒度的代码上下文；
3. 设计不同类型的 Prompt；
4. 调用大语言模型完成 Merge Prediction 任务；
5. 调用大语言模型生成代码审查意见；
6. 比较不同上下文和 Prompt 设计对模型性能的影响。

4.7 实验步骤

4.7.1 步骤一：数据准备

读取实验三中的测试数据，并构建代码审查任务所需的数据输入。

4.7.2 步骤二：构建代码上下文

分别构建以下不同类型的上下文：

- 仅包含 Diff；
- Diff + Pull Request 描述；
- Diff + Commit Message；
- Diff + 等其他信息。

4.7.3 步骤三：Prompt 设计

针对每种上下文分别设计：

- Zero-shot Prompt；
- Few-shot Prompt；
- Chain-of-Thought Prompt；
- Role-based Prompt。

4.7.4 步骤四：模型推理

调用大语言模型分别完成：

- Merge Prediction;
- Review Comment Generation。

记录模型输出及推理时间。

4.7.5 步骤五：结果分析

统计不同上下文和 Prompt 设计下模型性能，包括但不限于：

- Accuracy;
- Precision;
- Recall;
- F1-score;
- BLEU;
- ROUGE;
- 推理时间。

比较不同 Prompt 设计和上下文组合的优缺点。

4.8 实验结果

实验报告中，学生应展示以下实验结果：

1. 不同代码上下文构建样例；
2. 不同 Prompt 设计样例；
3. Merge Prediction 实验结果；
4. 自动生成的代码审查意见样例；
5. 不同 Prompt 设计性能比较；
6. 不同代码上下文性能比较；
7. 与实验三深度学习模型的性能对比分析。

4.9 实验思考

请结合实验过程回答以下问题：

- 1. 为什么代码上下文会影响大语言模型的推理能力？
- 2. Few-shot Prompt 相比 Zero-shot Prompt 有哪些优势？
- 3. 哪类上下文对 Merge Prediction 贡献最大？为什么？
- 4. 不同 Prompt 设计为什么会生成不同质量的代码审查意见？
- 5. 大语言模型相比实验三中的深度学习模型有哪些优势与不足？

4.10 实验拓展

完成基础实验后，可进一步尝试：

- 探索 Repository 级上下文、跨文件上下文对代码审查性能的影响；
- 尝试 Self-Consistency、Self-Reflection 等高级 Prompt 策略；
- 比较不同大语言模型（如 Qwen、DeepSeek、GPT 等）在代码审查任务上的表现，为后续 AI 生成代码的代码审查实验提供基准。

5 实验五 针对大模型生成代码的代码审查

5.1 实验基本信息

实验名称	针对大模型生成代码的代码审查
实验学时	4 学时
实验类型	综合性实验
实验性质	必修
教学方式	讲授 + 实践
对应课程目标	T1、T2、T3

5.2 实验目的

完成本实验后，学生应能够：

- 1. 理解 AI 生成代码与人类编写代码之间的差异。
- 2. 掌握 AI 生成代码数据集的构建与筛选方法。

3. 能够利用实验二、实验三和实验四构建的代码审查模型分析 AI 生成代码。
4. 掌握不同代码审查模型在 AI 生成代码上的性能评估方法。
5. 理解 AI 生成代码给代码审查带来的新挑战。
6. 能够分析不同模型在 AI 生成代码场景下的优缺点。

5.3 实验背景

随着 GitHub Copilot、ChatGPT、Claude、Cursor 等大语言模型辅助编程工具的广泛应用，越来越多的软件代码由 AI 辅助甚至完全自动生成。相比传统的人类编写代码，AI 生成代码在代码风格、程序结构、逻辑组织以及潜在缺陷等方面具有明显差异。

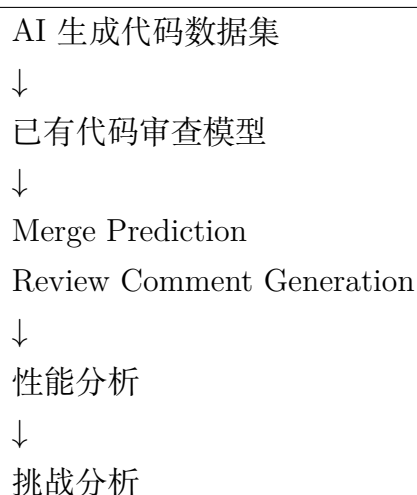
实验二、实验三和实验四分别利用传统机器学习、深度学习以及大语言模型构建了针对人类编写代码的代码审查模型。然而，这些模型均是在人类代码数据集上训练或评估得到，其在 AI 生成代码上的性能尚不明确。

本实验将在实验一构建的数据集中筛选 AI 生成代码对应的 Pull Request，并直接运行前述实验中已经构建好的代码审查工具，分析不同模型在 AI 生成代码场景下的性能变化，理解 AI 生成代码给代码审查带来的新挑战，并为后续设计新的代码审查方法提供依据。

5.4 实验原理

5.4.1 AI 生成代码代码审查流程

针对 AI 生成代码的代码审查流程如下：



本实验不重新训练模型，而是分析已有模型在新场景中的泛化能力。

5.4.2 AI 生成代码特点

相比人类编写代码，AI 生成代码通常具有以下特点：

- 代码风格更加统一；
- 重复性代码较多；
- 局部实现正确但整体逻辑存在缺陷；
- 容易产生幻觉（Hallucination）；
- 更依赖上下文信息完成实现。

这些特点使传统代码审查方法可能难以准确识别代码质量问题。

5.4.3 模型泛化能力分析

本实验主要比较以下三类模型：

- 实验二构建的传统机器学习模型；
- 实验三构建的深度学习模型；
- 实验四构建的大语言模型。

比较三类模型在人类代码与 AI 生成代码上的性能变化，分析模型的泛化能力。

5.4.4 代码审查任务

本实验仍关注两个典型任务：

1. Merge Prediction

预测 AI 生成代码对应的 Pull Request 是否能够被项目接受。

2. Review Comment Generation

自动生成针对 AI 生成代码的代码审查意见，并分析生成质量。

5.5 实验环境

5.5.1 硬件环境

建议配置如下：

- CPU：8 Core 以上；

- 内存：16GB 以上；
- GPU：推荐 NVIDIA RTX3060 及以上；
- 网络环境：能够访问所使用的大语言模型服务。

5.5.2 软件环境

建议软件版本如下：

软件	推荐版本
Python	3.10+
PyTorch	最新版本
Transformers	最新版本
scikit-learn	最新版本
pandas	最新版本
numpy	最新版本
matplotlib	最新版本

5.6 实验内容

本实验需要完成以下任务：

1. 从实验一构建的数据集中筛选 AI 生成代码对应的 Pull Request；
2. 利用实验二构建的传统机器学习模型进行代码审查；
3. 利用实验三构建的深度学习模型进行代码审查；
4. 利用实验四构建的大语言模型进行代码审查；
5. 比较不同模型在人类代码和 AI 生成代码上的性能变化；
6. 分析 AI 生成代码给代码审查带来的新挑战。

5.7 实验步骤

5.7.1 步骤一：数据筛选

读取实验一构建的数据集，仅保留 AI 生成代码对应的 Pull Request，并完成数据统计。

5.7.2 步骤二：传统机器学习模型测试

利用实验二训练完成的 SVM 和 Random Forest 模型，对 AI 生成代码数据进行 Merge Prediction，并记录实验结果。

5.7.3 步骤三：深度学习模型测试

利用实验三训练完成的 Code2Vec 和 CodeBERT 模型，对 AI 生成代码进行 Merge Prediction 和 Review Comment Generation。

5.7.4 步骤四：大语言模型测试

利用实验四设计好的 Prompt 及代码上下文，调用大语言模型完成 AI 生成代码的代码审查任务。

5.7.5 步骤五：性能分析

统计不同模型在人类代码和 AI 生成代码上的实验结果，包括但不限于：

- Accuracy;
- Precision;
- Recall;
- F1-score;
- BLEU;
- ROUGE。

进一步分析模型性能下降的原因以及 AI 生成代码带来的挑战。

5.8 实验结果

实验报告中，学生应展示以下实验结果：

1. AI 生成代码数据集统计结果；
2. 不同模型在 AI 生成代码上的 Merge Prediction 结果；
3. 自动生成代码审查意见样例；
4. 人类代码与 AI 生成代码实验结果对比；
5. 不同模型性能变化分析；
6. AI 生成代码代码审查面临的新挑战分析。

5.9 实验思考

请结合实验过程回答以下问题：

- 1. AI 生成代码与人类编写代码有哪些本质区别？
- 2. 为什么已有代码审查模型在 AI 生成代码上的性能可能下降？
- 3. 哪类模型对 AI 生成代码具有更好的泛化能力？为什么？
- 4. AI 生成代码是否需要新的评价指标？为什么？
- 5. 本实验发现了哪些 AI 生成代码特有的代码审查挑战？

5.10 实验拓展

完成基础实验后，可进一步尝试：

- 分析不同大语言模型（如 GPT、Claude、Qwen、DeepSeek）生成代码之间的差异；
- 探索不同类型 AI 生成代码（补全、重构、自动修复等）的代码审查特点；
- 总结 AI 生成代码特有的问题，为实验六设计针对性的代码审查方法提供依据。

6 实验六 改进针对大模型生成代码的代码审查

6.1 实验基本信息

实验名称	改进针对大模型生成代码的代码审查
实验学时	4 学时
实验类型	综合性实验
实验性质	必修
教学方式	讲授 + 实践
对应课程目标	T1、T2、T3

6.2 实验目的

完成本实验后，学生应能够：

- 1. 理解 AI 生成代码代码审查性能下降的主要原因。
- 2. 掌握不同代码上下文构建方法及其对代码审查任务的影响。

3. 掌握 Prompt Engineering 优化代码审查性能的方法。
4. 能够设计适用于 AI 生成代码的 Prompt。
5. 能够分析不同 Prompt 和上下文组合对 Merge Prediction 和 Review Comment Generation 任务的影响。
6. 能够提出针对 AI 生成代码的代码审查改进方案。

6.3 实验背景

实验五分析了传统机器学习模型、深度学习模型以及大语言模型在人类代码和 AI 生成代码上的性能差异。实验结果表明，AI 生成代码具有不同于人类代码的特征，使得已有代码审查方法在 Merge Prediction 和 Review Comment Generation 任务上的性能出现下降。

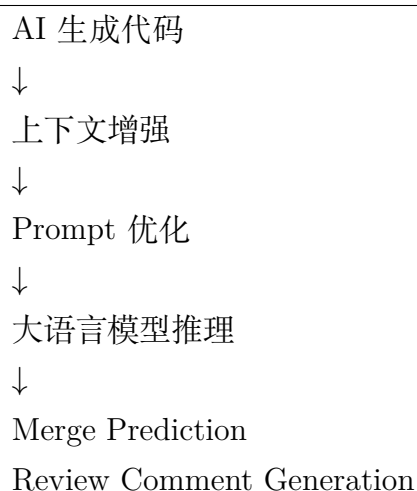
与传统软件工程任务不同，AI 生成代码通常更加依赖完整的软件工程上下文，仅利用 Pull Request 中的局部修改往往无法准确判断代码质量。同时，大语言模型具有较强的上下文理解能力，其性能受到 Prompt 设计方式和上下文组织方式的显著影响。

因此，本实验将在实验五的基础上，通过设计不同粒度的软件工程上下文以及不同类型的 Prompt，对 AI 生成代码的代码审查方法进行改进，提高 Merge Prediction 和 Review Comment Generation 任务的性能。

6.4 实验原理

6.4.1 改进代码审查流程

针对 AI 生成代码的改进代码审查流程如下：



相比实验五，本实验重点优化模型输入，而不是修改模型本身。

6.4.2 上下文增强

为了帮助模型更全面地理解代码修改，本实验考虑引入更加丰富的软件工程上下文，包括：

- Pull Request 描述；
- Commit Message；
- 修改前后的代码；
- 修改函数所在文件；
- 调用关系及相关函数；
- Issue 描述；
- 历史代码审查意见；
- Repository 级代码上下文。

不同上下文能够提供不同层次的信息，从而提高模型的推理能力。

6.4.3 Prompt 优化

Prompt 设计直接影响大语言模型的推理质量。本实验主要探索以下优化策略：

- Role-based Prompt；
- Few-shot Prompt；
- Chain-of-Thought Prompt；
- Self-Reflection Prompt；
- 多轮交互式 Prompt。

通过不同 Prompt 引导模型完成更加准确的代码审查。

6.4.4 代码审查任务

本实验继续关注以下两个任务：

1. Merge Prediction

结合增强后的上下文预测 AI 生成代码是否能够成功 Merge。

2. Review Comment Generation

生成更加准确、更加具有指导意义的代码审查意见。

6.5 实验环境

6.5.1 硬件环境

建议配置如下：

- CPU：8 Core 以上；
- 内存：16GB 以上；
- GPU：推荐 NVIDIA RTX3060 及以上；
- 网络环境：能够访问所使用的大语言模型服务。

6.5.2 软件环境

建议软件版本如下：

软件	推荐版本
Python	3.10+
Transformers	最新版本
OpenAI SDK 或兼容 SDK	最新版本
pandas	最新版本
numpy	最新版本
matplotlib	最新版本

6.6 实验内容

本实验需要完成以下任务：

1. 读取实验五中的 AI 生成代码数据集；
2. 构建不同粒度的软件工程上下文；
3. 设计多种 Prompt 优化策略；
4. 调用大语言模型完成 Merge Prediction；
5. 调用大语言模型生成代码审查意见；
6. 比较不同上下文与 Prompt 组合的性能，并分析改进效果。

6.7 实验步骤

6.7.1 步骤一：数据准备

读取实验五中的 AI 生成代码数据，并准备代码审查任务所需的输入。

6.7.2 步骤二：上下文构建

分别构建不同粒度的软件工程上下文，包括：

- Diff;
- Diff + Pull Request 描述;
- Diff + Repository 上下文;
- Diff + Issue 信息;
- 完整软件工程上下文。

6.7.3 步骤三：Prompt 设计

针对不同上下文设计以下 Prompt：

- Role-based Prompt;
- Few-shot Prompt;
- Chain-of-Thought Prompt;
- Self-Reflection Prompt。

6.7.4 步骤四：模型推理

利用设计好的上下文和 Prompt 调用大语言模型，分别完成：

- Merge Prediction;
- Review Comment Generation。

记录模型输出结果及推理时间。

6.7.5 步骤五：结果分析

统计不同方法的实验结果，包括但不限于：

- Accuracy;
- Precision;
- Recall;
- F1-score;
- BLEU;
- ROUGE;
- 推理时间。

分析不同上下文和 Prompt 设计对代码审查性能的影响，并与实验五进行对比。

6.8 实验结果

实验报告中，学生应展示以下实验结果：

1. 不同上下文构建样例；
2. 不同 Prompt 设计样例；
3. Merge Prediction 实验结果；
4. 自动生成代码审查意见样例；
5. 不同上下文性能比较；
6. 不同 Prompt 设计性能比较；
7. 与实验五实验结果的对比分析。

6.9 实验思考

请结合实验过程回答以下问题：

1. 为什么 Repository 级上下文能够提升代码审查性能？
2. 哪种 Prompt 设计最适合 AI 生成代码的代码审查？为什么？
3. Prompt 优化与上下文增强分别解决了哪些问题？
4. 不同 Prompt 组合是否适用于所有代码审查任务？
5. 针对 AI 生成代码，未来还可以从哪些方面进一步提升代码审查效果？

6.10 实验拓展

完成基础实验后，可进一步尝试：

- 尝试 Agent、多智能体等代码审查框架；
- 探索自动 Prompt 优化与 Prompt 搜索方法；
- 引入工具调用（Tool Use）、检索增强生成（RAG）等技术，提高 AI 生成代码代码审查的准确率，为实验七开发 VSCode 插件提供更加稳定、高质量的代码审查能力。

7 实验七 实现 VSCode 插件

7.1 实验基本信息

实验名称	实现 VSCode 插件
实验学时	4 学时
实验类型	综合性实验
实验性质	必修
教学方式	讲授 + 实践
对应课程目标	T1、T2、T3

7.2 实验目的

完成本实验后，学生应能够：

1. 理解 VSCode 插件的基本架构及开发流程。
2. 掌握 VSCode Extension API 的基本使用方法。
3. 能够设计智能代码审查插件的人机交互界面。
4. 能够将已有代码审查模型集成到 VSCode 插件中。
5. 能够实现代码审查结果的可视化展示。
6. 理解智能软件工具从算法验证到工程部署的完整过程。

7.3 实验背景

经过实验二至实验六，已经分别完成了传统机器学习、深度学习以及大语言模型代码审查模型的构建，并针对 AI 生成代码进行了性能分析与优化。然而，这些模型主要以脚本程序的形式运行，距离真实的软件开发环境仍存在一定差距。

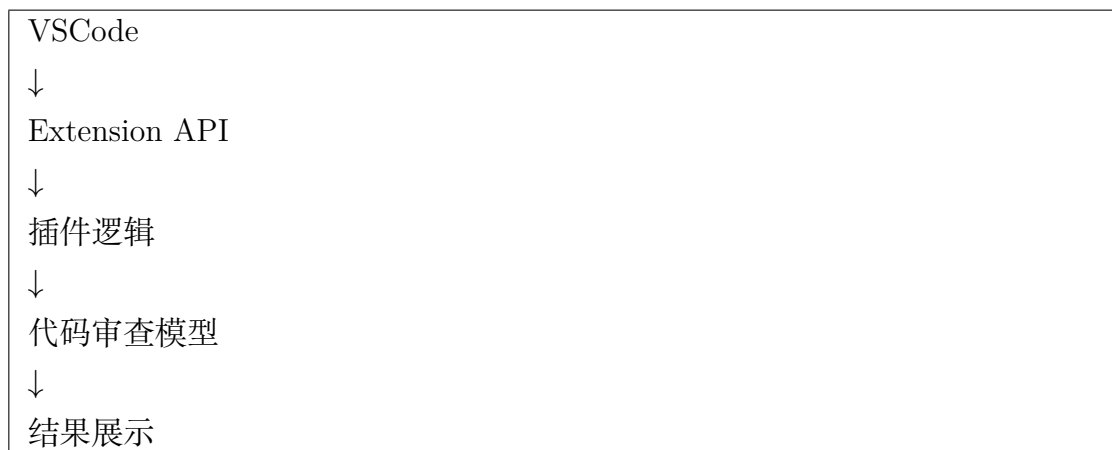
VSCode 作为目前最广泛使用的软件开发工具之一，提供了完善的插件开发接口，使开发者能够将智能分析能力直接集成到软件开发过程中，实现开发、代码审查和反馈的一体化。

本实验将在前六个实验实现的代码审查模型基础上，开发一个 VSCode 智能代码审查插件，使开发者能够在编写代码过程中实时调用代码审查模型，预测 Pull Request 是否能够被合并，并自动生成代码审查意见，从而提高软件开发效率。

7.4 实验原理

7.4.1 VSCode 插件架构

VSCode 插件通常包括以下几个组成部分：



插件通过 Extension API 获取编辑器中的代码信息，并调用后端模型完成智能代码审查。

7.4.2 插件功能设计

本实验设计的 VSCode 插件主要包括以下功能：

- 获取当前编辑文件；
- 获取代码修改内容；
- 调用代码审查模型；
- 展示 Merge Prediction 结果；

- 展示自动生成的代码审查意见；
- 查看历史代码审查记录。

7.4.3 模型集成方式

为了便于模型部署，本实验采用前后端分离方式：

- VSCode 插件负责用户交互；
- Python 后端负责调用机器学习、深度学习及大语言模型；
- 前后端通过 HTTP 接口进行通信。

这种架构能够方便模型更新，同时保证插件具有良好的扩展性。

7.4.4 结果可视化

为了帮助开发者快速理解代码质量，本实验设计以下展示方式：

- Merge Prediction 结果；
- 代码审查意见列表；
- 风险等级提示；
- 高亮显示存在问题的代码位置；
- 模型推理时间及调用信息。

7.5 实验环境

7.5.1 硬件环境

建议配置如下：

- CPU：8 Core 以上；
- 内存：16GB 以上；
- 网络环境：能够访问模型服务。

7.5.2 软件环境

建议软件版本如下：

软件	推荐版本
VSCode	最新版本
Node.js	最新版本
TypeScript	最新版本
Python	3.10+
Flask/FastAPI	最新版本
PyTorch	最新版本
Transformers	最新版本

7.6 实验内容

本实验需要完成以下任务：

1. 设计 VSCode 插件整体架构；
2. 实现插件基本功能；
3. 集成实验二至实验六开发的代码审查模型；
4. 实现插件与后端模型通信；
5. 实现代码审查结果可视化；
6. 完成插件测试与功能验证。

7.7 实验步骤

7.7.1 步骤一：插件项目创建

创建 VSCode Extension 项目，完成插件基本框架搭建，包括插件注册、菜单配置及命令定义。

7.7.2 步骤二：插件界面设计

设计插件交互界面，实现：

- 工具栏入口；
- 侧边栏页面；

- 审查结果展示页面；
- 配置页面。

7.7.3 步骤三：模型接口开发

封装实验二至实验六构建的代码审查模型，设计统一 HTTP 接口，实现：

- Merge Prediction 接口；
- Review Comment Generation 接口；
- 模型状态检测接口。

7.7.4 步骤四：插件功能集成

在 VSCode 插件中调用模型接口，实现：

- 获取当前代码；
- 自动发送代码审查请求；
- 获取模型返回结果；
- 展示代码审查意见。

7.7.5 步骤五：功能测试

对插件进行完整测试，包括但不限于：

- 插件安装测试；
- 模型调用测试；
- Merge Prediction 测试；
- Review Comment Generation 测试；
- 不同规模代码文件测试。

7.8 实验结果

实验报告中，学生应展示以下实验结果：

1. VSCode 插件整体架构图；
2. 插件主要功能界面截图；
3. 模型调用流程截图；
4. Merge Prediction 功能展示；
5. 自动生成代码审查意见展示；
6. 插件运行效果及性能分析。

7.9 实验思考

请结合实验过程回答以下问题：

1. 为什么代码审查工具需要集成到 IDE 中？
2. 前后端分离架构相比本地部署有哪些优势？
3. 如何提高插件的响应速度和用户体验？
4. 插件部署过程中遇到了哪些工程问题？如何解决？
5. 如果进一步完善该插件，还可以增加哪些功能？

7.10 实验拓展

完成基础实验后，可进一步尝试：

- 支持 GitHub Pull Request 在线代码审查；
- 增加多模型切换与模型配置功能；
- 实现代码修改建议自动应用；
- 集成 Agent、多智能体协同代码审查框架；
- 将插件扩展到 JetBrains IDE、Visual Studio 等其他开发环境，实现跨平台智能代码审查工具。